

Neu

A Polyglot Architecture Connecting Computational Theory, Language Design, and Participatory Biomedical Infrastructure

Erick Olusoji Oduniyi

Intelligent-Interfaces Group

2026

Abstract

We present NEU, a polyglot software architecture that connects three layers of computational infrastructure: AIE (ai expressions), a theoretical framework for classifying computational processes by their information-theoretic behavior; a language design synthesizing features from OCaml, TypeScript, and R; and CGEN-DLANG (Community Generated Design Language), a participatory evolutionary co-design framework. The architecture places TypeScript at the center of a polyglot system, with OCaml providing the formal kernel (type safety, module functors, exhaustive validation) and R providing the analytical layer (data frames, statistical modeling, vectorized computation). We implement this architecture as a microfrontend pipeline of composable Web Component modules, each classified by AIE engine type (synthesis, analysis, transformation) and language layer. We demonstrate the system in the domain of equitable biomedicine, where communities participate in drug discovery and disease spectroscopy while retaining data sovereignty, provenance tracking, and compensation rights. The contribution is threefold: (1) a formal connection between information-theoretic process classification and programming language type systems, (2) a concrete polyglot architecture that leverages each language's strengths without requiring a new compiler, and (3) a microfrontend module system where visual interface components are typed computational engines with traceable data flow.

Contents

1	Introduction	3
2	Mathematical Foundations	4
2.1	The Data Processing Inequality and Engine Classification	4
2.2	The Information Bottleneck and Pipeline Design	4
2.3	Categorical Semantics of Signal Flow	5
2.4	Applied Category Theory: Compositionality and Open Systems	5
2.5	Semantic Entropy and Communication	6
2.6	Summary of Mathematical Connections	6
3	AI Expressions: A Theoretical Framework	6
3.1	Engines	6
3.1.1	Synthesis Engines	7
3.1.2	Analysis Engines	7
3.1.3	Transformations	7
3.2	Functions and Operations	7
3.3	Applications	7
3.4	Environments	8
3.5	Expressions	8
4	The Neu Language Design	8
4.1	Type System	8
4.2	Data as a Primitive	9
4.3	Effect System	9
4.4	Module System	10
5	The Polyglot Architecture: OCaml–TypeScript–R	10
5.1	Design Decision: TypeScript in the Middle	10
5.2	OCaml to TypeScript via Melange	11
5.3	TypeScript to R via Arrow and WebR	11
5.4	What Each Language Handles	11
5.5	OCaml’s Compilation Reach: From FPGA to Browser	12
5.6	Svelte as the Rendering Layer	12
5.7	The Delegation Principle	13
6	cgen-dlang: Participatory Evolutionary Co-Design	13
6.1	Core Abstractions	14
6.2	cgen-dlang as an AIE Application	14
6.3	Domain Parameterization	14
6.4	Provenance as Infrastructure	15
7	Microfrontend Module System	15
7.1	Module Architecture	15
7.2	Pipeline Composition	15
7.3	Implemented Modules	16
7.4	Visual Design	16

8	Application: Equitable Biomedicine	16
8.1	The Detection Pipeline	16
8.2	Extending cgen-dlang to Drug Discovery	17
8.3	Governance Principles	17
8.4	The Full Stack	17
9	Related Work	18
9.1	Polyglot and Multi-Language Systems	18
9.2	Effect Systems and Typed Side Effects	18
9.3	Participatory Design in Computing	18
9.4	Evolutionary and Collective Intelligence	18
9.5	Closest Language Designs	19
9.6	AI-Assisted Compiler Construction and Self-Hosting	19
9.7	OCaml for Hardware Description	19
9.8	Compiler-Driven UI Frameworks	20
10	Conclusion	20

1 Introduction

Programming languages are typically designed in isolation: a type system here, a data model there, a concurrency primitive somewhere else. Practitioners then stitch these languages together through foreign function interfaces, serialization boundaries, and build system complexity. The result is that the theoretical properties a language guarantees—type safety, effect tracking, memory safety—dissolve at the boundary between systems.

This paper describes NEU, an architecture that takes a different approach. Rather than designing a single new language (though we sketch one), we connect three existing languages—OCaml, TypeScript, and R—through a shared theoretical framework called AIE (ai expressions) and a shared application framework called CGEN-DLANG (Community Generated Design Language). Each language handles what it does best:

OCaml

provides the formal kernel: algebraic data types, exhaustive pattern matching, module functors, and type inference. It defines the grammar of valid computations.

TypeScript

provides the orchestration hub: async coordination, web platform access, the application layer, microfrontend composition, and the evolutionary co-design framework.

R provides the analytical layer: first-class data frames, vectorized operations, statistical modeling, and visualization.

The connecting tissue is AIE, which classifies every computational process as one of three engine types based on information-theoretic behavior:

- **Synthesis engines:** processes that increase information content ($H(S_n) > H(S_1)$)
- **Analysis engines:** processes that decrease information content ($H(S_n) < H(S_1)$)
- **Transformations:** processes that preserve information content ($H(S_n) = H(S_1)$)

This classification is not merely taxonomic. When embedded in a type system, it becomes a compile-time property: the system can verify that a pipeline stage labeled “analysis” actually reduces information, that a “transformation” does not silently drop data, and that an “application” (which composes both synthesis and analysis) maintains the governance properties required by its domain.

We demonstrate the architecture in the domain of equitable biomedicine—specifically, participatory drug discovery and disease spectroscopy—where the formal properties matter concretely: data sovereignty requires that patient data never leaves its origin without tracked consent, attribution requires provenance at every pipeline stage, and community governance requires that the fitness function driving evolutionary optimization is shaped by the people affected by the outcomes.

The paper is organized as follows. Section 2 establishes the mathematical foundations connecting information theory, category theory, and graph signal processing to the AIE framework. Section 3 presents the AIE theoretical framework. Section 4 describes the NEU language design and its mapping to existing languages. Section 5 details the polyglot architecture with OCaml, TypeScript, and R, including the delegation principle, OCaml’s compilation reach from FPGA hardware to browser UI, and the role of compiler-driven frameworks like Svelte. Section 6 describes the CGEN-DLANG participatory framework. Section 7 presents the microfrontend module system. Section 8

demonstrates the application to equitable biomedicine. Section 9 discusses related work, and Section 10 concludes.

2 Mathematical Foundations

The AIE framework draws on three mathematical traditions: information theory, category theory, and graph signal processing. This section identifies the formal structures that underpin the engine classification and connects them to existing mathematical results.

2.1 The Data Processing Inequality and Engine Classification

The classification of engines by entropy behavior is grounded in a fundamental result of information theory: the **Data Processing Inequality** (DPI). For a Markov chain $X \rightarrow Y \rightarrow Z$, the DPI states:

$$I(X; Z) \leq I(X; Y) \quad (1)$$

where $I(\cdot; \cdot)$ denotes mutual information. Equivalently, processing cannot increase the information that a signal carries about its source [20]. This is the formal basis for AIE’s analysis engines: any deterministic function applied to a random variable can only decrease or preserve entropy, never increase it.

Synthesis engines appear to violate the DPI, but the resolution is that they introduce *new* information—from external sources, random number generators, or learned generative models. A synthesis engine E_S with access to an external source W satisfies:

$$H(E_S(S, W)) > H(S) \quad \text{when} \quad I(W; S) < H(W) \quad (2)$$

The new information comes from W , not from processing S alone. In the CGEN-DLANG framework, W is the community’s votes and the stochastic operators (mutation, crossover). In the biomedical pipeline, W is the molecular dynamics simulation or the AlphaFold structure prediction.

Transformations are the special case where processing is *invertible*: a bijective function $f : S \rightarrow S'$ satisfies $H(S') = H(S)$. Format conversions, lossless compression, and grammar validation (which either passes or rejects, but does not alter the candidate) are transformations.

2.2 The Information Bottleneck and Pipeline Design

Tishby, Pereira, and Bialek’s **Information Bottleneck** (IB) method [21] provides a formal framework for the tradeoff between compression and prediction. Given an input X and a target Y , the IB seeks a compressed representation T that minimizes $I(X; T)$ (compression) while maximizing $I(T; Y)$ (prediction):

$$\min_{p(t|x)} [I(X; T) - \beta I(T; Y)] \quad (3)$$

Shwartz-Ziv and Tishby [22] applied this to deep neural networks, showing that training proceeds in two phases: a *fitting phase* where mutual information $I(T; Y)$ increases (the network learns to predict), followed by a *compression phase* where $I(X; T)$ decreases (the network discards irrelevant input features). This two-phase structure maps directly onto AIE’s engine classification:

- The **fitting phase** is a synthesis engine: the network’s internal representation gains information about the target.

- The **compression phase** is an analysis engine: the representation sheds information about the input that is irrelevant to the target.

The disease spectroscopy pipeline exhibits the same structure. The measurement stage captures a high-dimensional spectral signal (fitting/synthesis relative to the clinical question). The classification stage compresses that signal to a risk score (compression/analysis). The IB framework provides the mathematical criterion for evaluating whether the pipeline achieves the optimal tradeoff between retaining clinically relevant biomarker information and discarding noise.

2.3 Categorical Semantics of Signal Flow

Bonchi, Sobociński, and Zanasi [23] developed a categorical semantics for signal flow graphs, showing that linear signal processing systems can be formalized as morphisms in a symmetric monoidal category. Their key result: signal flow graphs compose according to the algebraic laws of a PROP (products and permutations category), and the denotational semantics (what the system computes) can be related to operational semantics (how it executes) through categorical equivalence.

This is directly relevant to AIE’s engine composition. When we write $E = f_1 \rightarrow \dots \rightarrow f_n$, we are composing morphisms in a category where:

- Objects are state types (the data flowing through the pipeline)
- Morphisms are engines (typed by their information-theoretic behavior)
- Composition is sequential pipeline connection
- The monoidal product is parallel execution

The engine classification (synthesis/analysis/transformation) then becomes a *grading* on the category: a functor from the pipeline category to an ordered monoid $(\{-1, 0, +1\}, +)$ where $+1$ denotes synthesis, -1 denotes analysis, and 0 denotes transformation. The grade of a composed pipeline is the sum of its stages’ grades, giving a coarse but useful characterization of the pipeline’s net information behavior.

2.4 Applied Category Theory: Compositionality and Open Systems

Fong and Spivak’s *Seven Sketches in Compositionality* [24] provides the broader mathematical context. Their treatment of applied category theory covers:

- **Co-design via profunctors** (Chapter 4): formalizes how components with specified interfaces compose into systems. This is the mathematical structure underlying our microfrontend module system, where modules declare typed input and output ports.
- **Signal flow graphs via PROPs** (Chapter 5): formalizes the algebraic structure of signal processing pipelines, directly applicable to the spectroscopy pipeline.
- **Collaborative design via enriched categories**: formalizes how multiple agents (community members in CGEN-DLANG) contribute to a shared design space with quantifiable preferences.

Foley, Breiner, Subrahmanian, and Dusel [25] extend this to complex system design using **operads**—algebraic structures that formalize how operations with multiple inputs and one output compose. Their CASCADE framework uses operads for “complex system design specification, analysis and synthesis”—notably, the same three terms that AIE uses for its engine classification. In their framework, operads provide the grammar for valid system compositions, while operad

algebras provide the semantics. This parallels CGEN-DLANG’s component grammar (the operad) and component library (the algebra).

2.5 Semantic Entropy and Communication

Recent work on semantic communication [26] introduces a category-theoretic framework for **semantic entropy**—entropy measured not over raw symbols but over their meanings relative to a knowledge base. This connects to AIE’s concept of “semantic syncing” (Chapter 5 of the ai expressions book): the act of aligning two representational systems. In the categorical framework, semantic entropy reduction is achieved by exploiting dependencies captured in a knowledge base, formalized as a functor from a syntactic category to a semantic category.

For the biomedical application, this means that the “information” in AIE’s engine classification is not merely Shannon entropy over bit strings, but *semantic* information relative to the clinical domain. A spectral measurement that is high-entropy in the signal-processing sense may be low-entropy in the clinical sense if it unambiguously indicates a specific biomarker. The engine classification should ultimately be graded by semantic entropy, not raw Shannon entropy—a direction for future formalization.

2.6 Summary of Mathematical Connections

Mathematical Structure	Connection to aie/Neu	Source
Data Processing Inequality	Analysis engines cannot increase information; synthesis requires external source	Cover & Thomas
Information Bottleneck	Pipeline stages trade off compression vs. prediction; fitting = synthesis, compression = analysis	Tishby et al.
Categorical signal flow (PROPs)	Engine composition is morphism composition in a symmetric monoidal category	Bonchi et al.
Operads for system design	Component grammars are operads; component libraries are operad algebras	Foley et al.
Profunctors for co-design	Module ports and typed connections formalize as profunctors between interface categories	Fong & Spivak
Semantic entropy	Engine classification should be graded by semantic, not raw Shannon, entropy	Yang et al.

Table 1: Mathematical structures underlying the AIE framework.

3 AI Expressions: A Theoretical Framework

AIE (ai expressions) is a theoretical framework for describing and manipulating **process architectures** (engines), **functions** (flows), and **operations** (orchestras). It is derived from category theory, information theory, and graph signal processing. This section formalizes the core constructs.

3.1 Engines

An engine is a process or sequence of functions applied to a state S . Engines are classified by their effect on the information content (Shannon entropy H) of the state.

3.1.1 Synthesis Engines

A synthesis engine is a process that creates or adds information:

$$E_S : f_1 \rightarrow \cdots \rightarrow f_n \in \{S_n, S_1 \mid H(S_n) > H(S_1)\} \quad (4)$$

Examples include: generating molecular candidates, composing treatment protocols, creating spectral models from molecular dynamics simulations, and producing novel designs from evolutionary crossover and mutation.

3.1.2 Analysis Engines

An analysis engine is a process that removes or subtracts information:

$$E_A : f_1 \rightarrow \cdots \rightarrow f_n \in \{S_n, S_1 \mid H(S_n) < H(S_1)\} \quad (5)$$

Examples include: spectroscopic detection (reducing a biological sample to a risk score), feature extraction, fitness evaluation, community voting and selection, and ML classification.

3.1.3 Transformations

A transformation is a process that preserves information:

$$T : f_1 \rightarrow \cdots \rightarrow f_n \in \{S_n, S_1 \mid H(S_n) = H(S_1)\} \quad (6)$$

Examples include: federated learning aggregation (reshaping model weights without losing patient information), data format conversions, provenance logging, and grammar validation.

3.2 Functions and Operations

Functions (flows) are mathematical functions that compose into engines. Any function $f : A \rightarrow B$ can serve as a stage in an engine pipeline.

Operations are unary, binary, or ternary operations applied within and across functions. In the CGEN-DLANG framework, the evolutionary operators `compare()`, `mutate()`, and `cross()` are operations.

3.3 Applications

An **application** is a system that utilizes both synthesis and analysis engines. Applications are the first composite structure in AIE:

$$\text{Application} = \{E_{S_1}, \dots, E_{S_m}\} \cup \{E_{A_1}, \dots, E_{A_k}\} \quad (7)$$

The CGEN-DLANG framework is an application: it synthesizes new candidates (mutation, crossover) and analyzes them (fitness evaluation, community voting, selection).

3.4 Environments

An **environment** runs applications. Unlike the previous constructs, environments are not theoretical machines—they are resource-dependent systems. A hospital network running federated learning, a community data cooperative governing research access, or an HPC cluster performing molecular dynamics simulations are all environments in the AIE sense.

3.5 Expressions

The framework is called “ai expressions” because everything is an expression—every construct returns a value, every process has a classifiable information-theoretic signature, and the entire system is compositional. This mirrors the expression-oriented design of functional programming languages, which is not coincidental: the NEU language design makes this connection explicit.

4 The Neu Language Design

NEU is a language design that synthesizes features from five languages, selected for complementary strengths: JavaScript (ubiquity, async), R (data primitives, vectorized operations), Haskell (type system, effect tracking), Rust (memory safety, traits), and OCaml (module system, pragmatic FP). The design is guided by five principles:

1. **Expression-oriented**: everything returns a value (Haskell, OCaml, Rust).
2. **Data-first**: tabular data and vectors are language primitives, not libraries (R).
3. **Safe by default**: no nulls, no exceptions; algebraic error handling via `Result` and `Option` (Rust, Haskell).
4. **Pragmatically functional**: pure by default with explicit effects, but mutability is available (OCaml).
5. **Runs everywhere**: compiles to native, WebAssembly, and can be interpreted in a REPL (JavaScript’s reach, R’s exploratory workflow).

4.1 Type System

From Haskell and OCaml, NEU takes algebraic data types, parametric polymorphism, and type inference. From Rust, traits as practical typeclasses. From TypeScript, structural typing for record types.

```
// Algebraic data types
type Result<T, E> = Ok(T) | Err(E)
type Option<T> = Some(T) | None

// Structural record types
type User = { name: String, age: Int }

// Traits
trait Summarize {
  fn summary(self) -> String
}

impl Summarize for User {
```

```
fn summary(self) -> String = "{self.name}, age {self.age}"
}
```

The critical extension for AIE is that engine classification becomes a type-level property:

```
trait SynthesisEngine<S> {
  fn run(self, state: S) -> S // H(output) > H(input)
}

trait AnalysisEngine<S> {
  fn run(self, state: S) -> S // H(output) < H(input)
}
```

4.2 Data as a Primitive

From R, vectors and data frames are first-class. Vectorized operations and pipe operators are built into the language:

```
let df = dataframe {
  name: ["Alice", "Bob", "Carol"],
  score: [92, 87, 95],
}

let result = df
|> filter(.score > 90)
|> select(.name, .score)
|> mutate(curved = .score + 5)
```

4.3 Effect System

From Haskell, side effects are tracked in the type system. From JavaScript, `async/await` provides ergonomic syntax. The effect signature documents what a function does:

```
fn read_config(path: String) -> IO<Result<Config, FileError>> {
  let contents = await fs.read(path)?
  parse_config(contents)
}
```

For the biomedical domain, this means federated computation boundaries are type boundaries:

```
fn detect_dementia(sample: own BloodSample)
-> Federated<IO<Result<RiskScore, DetectionError>>> {
  sample
  |> measure_atr_ftir()
  |> preprocess_spectra()
  |> classify_risk()
  |> log_provenance()
  |> trigger_compensation()
}
```

The `own` keyword (from Rust) signals that the patient's data does not leave their control. `Federated<IO<...>>` signals distributed computation. `log_provenance()` and `trigger_compensation()` are effects visible in the type signature—they cannot be omitted.

4.4 Module System

From OCaml, NEU takes module functors—modules parameterized by other modules:

```
module MakeCgen(Domain: ComponentDomain) {
  fn evolve(population: Library, fitness: fn(Component) -> Score)
    -> IO<Library> {
    population
      |> evaluate(fitness)
      |> select_by_vote()
      |> mutate(Domain.grammar)
      |> cross(Domain.grammar)
  }
}

module BioCgen = MakeCgen(BioComponents)
```

This is how CGEN-DLANG becomes domain-agnostic: the same evolutionary framework operates over visual design assets, molecular structures, or spectroscopic biomarker signatures by swapping the domain module.

5 The Polyglot Architecture: OCaml–TypeScript–R

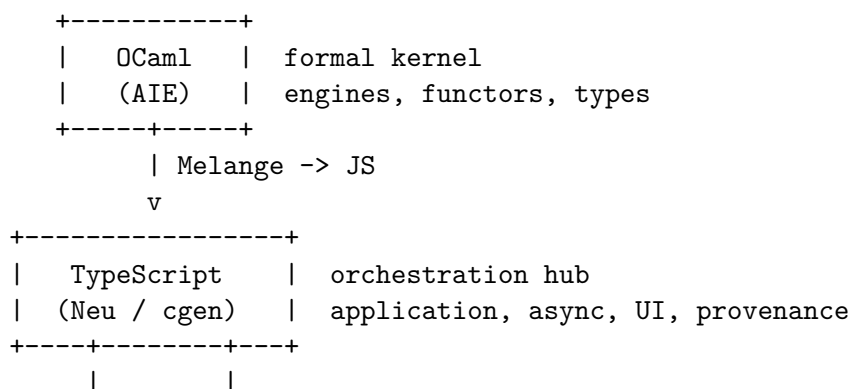
While NEU describes an ideal language, building a compiler is a multi-year effort. The polyglot architecture achieves approximately 80% of NEU’s guarantees today by distributing responsibilities across three existing languages, connected through well-defined interfaces.

5.1 Design Decision: TypeScript in the Middle

The architecture places TypeScript at the center, with OCaml and R at the periphery. This is a deliberate choice based on practical constraints:

- **OCaml in the middle** would require OCaml to handle web APIs, async orchestration, UI rendering, and general application plumbing—areas where its ecosystem is thin.
- **TypeScript in the middle** leverages the npm ecosystem for connectivity (WebR, Apache Arrow, Melange output, HTTP, WebAssembly, browser APIs) while letting OCaml and R focus on their strengths.

The resulting architecture:



```

Arrow|           | WebR / Plumber
      v           v
+-----+ +-----+
|   R   | |   R   | data analysis
|       | |       | statistics, spectral, viz
+-----+ +-----+

```

5.2 OCaml to TypeScript via Melange

Melange (formerly BuckleScript) compiles OCaml to readable JavaScript with clean interop. The OCaml layer defines:

- The AIE engine algebra (synthesis, analysis, transformation types)
- Module functors for domain parameterization
- Exhaustive pattern matching for grammar validation
- Type definitions that generate `.d.ts` declarations for TypeScript consumption

The compiled output is human-readable JavaScript that TypeScript imports with full type safety. OCaml's guarantees (exhaustive matching, type inference, functor correctness) are enforced at compile time in the OCaml layer and then consumed as typed interfaces in TypeScript.

5.3 TypeScript to R via Arrow and WebR

Two bridges connect TypeScript to R:

Apache Arrow

provides a columnar memory format that both TypeScript (`apache-arrow` npm package) and R (`arrow` R package) read and write natively. Data frames pass between languages with zero-copy semantics.

WebR

compiles R to WebAssembly, allowing R to run directly in Node.js or the browser. TypeScript calls R's statistical functions without a separate R process.

5.4 What Each Language Handles

Language	Responsibility	aie Role
OCaml	Engine algebra, module functors, grammar validation, type inference, pattern matching	Theory kernel
TypeScript	Application orchestration, CGEN-DLANG framework, async/effects, microfrontend UI, provenance tracking, evolutionary co-design	Application hub
R	Data frames, vectorized operations, statistical modeling, spectral analysis, cohort summaries, visualization	Analysis layer

Table 2: Language responsibilities in the polyglot architecture.

5.5 OCaml’s Compilation Reach: From FPGA to Browser

A distinctive property of OCaml in this architecture is the breadth of its compilation targets. The same language—with the same type system, module functors, and pattern matching—can target radically different substrates:

Native code

OCaml’s native compiler produces efficient machine code for backend services and CLI tools.

JavaScript (Melange)

As described above, Melange compiles OCaml to readable JavaScript for browser and Node.js consumption.

FPGA hardware (Hardcaml)

Hardcaml [17] is an OCaml-embedded DSL for Register Transfer Level (RTL) hardware design, developed and used in production at Jane Street. Engineers write OCaml code that compiles to synthesizable Verilog or VHDL for FPGA toolchains. Hardcaml provides cycle-accurate simulation, formal verification, and parametric hardware modules using OCaml’s functors.

This means OCaml can define the formal engine algebra *and* the hardware accelerators for compute-intensive pipeline stages. For the biomedical application, spectral preprocessing (FFT, baseline correction) and ML inference for the classification stage are exactly the kind of DSP workloads that FPGAs excel at. The same OCaml functors that parameterize the CGEN-DLANG domain could parameterize Hardcaml hardware modules:

```
OCaml
|-- Melange -> JS -> Svelte/TypeScript (UI)
|-- Native -> backend services
+-- Hardcaml -> Verilog -> FPGA
    |-- Spectral DSP (FFT, filtering)
    |-- ML inference (classification)
    +-- Cryptographic provenance
```

5.6 Svelte as the Rendering Layer

The microfrontend modules described in Section 7 are currently implemented as Web Components with manual Shadow DOM rendering. A natural evolution is to adopt Svelte [19] as the rendering layer. Svelte and OCaml share a core philosophy: **move decisions from runtime to compile time**. Svelte compiles declarative UI components into minimal imperative JavaScript that directly mutates the DOM—no virtual DOM, no runtime diffing. This mirrors OCaml’s approach of resolving type safety, exhaustive matching, and module resolution at compile time rather than runtime.

The integration is direct: Melange compiles OCaml to clean JavaScript; Svelte components import JavaScript. OCaml-compiled engine types and grammar validators become reactive stores consumed by Svelte components. The extended architecture becomes:

Layer	Technology	Role
Theory	AIE	Engine classification
Formal kernel	OCaml	Types, functors, grammar
Hardware	OCaml $\rightarrow$ Hardcaml $\rightarrow$ FPGA	Spectral DSP, ML inference
UI components	Svelte (consuming OCaml via Melange)	Reactive microfrontends
Orchestration	TypeScript	Pipeline wiring, async, CGEN-DLANG
Data analysis	R (via WebR/Arrow)	Statistics, visualization

Table 3: Extended architecture with Svelte and FPGA layers.

5.7 The Delegation Principle

A key architectural insight, validated by recent work in AI-assisted compiler construction [16], is that **correctness without optimization is insufficient when physical constraints apply**. Anthropic’s engineering team demonstrated an AI system that wrote a self-hosting C compiler targeting x86, ARM, and RISC-V. The compiler could generate correct 16-bit x86 code using 0x66/0x67 opcode prefixes, but the resulting binary exceeded 60KB—far beyond the 32KB limit Linux enforces for real-mode boot code. Rather than compromising correctness, the system delegated that specific phase to GCC. On ARM and RISC-V, where instruction encodings do not require the prefix workaround, the compiler handled everything end-to-end.

This is precisely the principle underlying our polyglot architecture. Each language in the stack handles what it does best, and delegates to specialized tools when a domain-specific constraint (type safety, statistical modeling, hardware synthesis, UI reactivity) is better served elsewhere:

- When grammar validation requires exhaustive pattern matching, delegate to OCaml.
- When statistical analysis requires vectorized data frame operations, delegate to R.
- When spectral DSP requires parallel hardware execution, delegate to FPGA via Hardcaml.
- When reactive UI rendering requires compile-time DOM optimization, delegate to Svelte.
- When pipeline orchestration requires async coordination and ecosystem connectivity, delegate to TypeScript.

The honest engineering move—admitting that a given phase is better handled by a specialized tool—is not a failure of the architecture. It *is* the architecture. The skill is in knowing where the boundaries are and designing clean, typed interfaces between them. The AIE framework provides the shared vocabulary (engine types, information-theoretic classification) that makes these boundaries legible across languages and compilation targets.

6 cgen-dlang: Participatory Evolutionary Co-Design

CGEN-DLANG (Community Generated Design Language) is a collaborative design framework for small communities (5–50 people) who want to generate structures that reflect collective aesthetic or functional values through evolutionary processes [1]. The framework is domain-agnostic: its abstractions parameterize over the component type.

6.1 Core Abstractions

Component Library

A collection of domain-specific primitives. In visual design: shapes, colors, textures. In biomedicine: molecular structures, biomarker signatures, drug compounds.

Component Grammar

Rules defining valid operations on and between components. In visual design: composition rules. In biomedicine: Lipinski’s Rule of Five, biochemical validity constraints, AlphaFold-validated structural viability.

Voting Model

A schema for how the community evaluates candidates. The voting model is the mechanism by which human judgment enters the evolutionary loop.

Evolutionary Operators

`compare()`, `mutate()`, and `cross()` drive iterative refinement. These are AIE operations applied within synthesis engines.

Fitness Function

The aggregation of community votes and domain-specific metrics. In CGEN-DLANG, the fitness function is collectively defined—it is not imposed by a single researcher or institution.

6.2 cgen-dlang as an AIE Application

In AIE terms, CGEN-DLANG is an application because it composes both synthesis and analysis engines:

- **Synthesis:** mutation and crossover generate new candidates (information increases)
- **Analysis:** fitness evaluation and selection reduce the population (information decreases)
- **Transformation:** grammar validation checks constraints without altering candidates (information preserved)

6.3 Domain Parameterization

The key architectural insight is that CGEN-DLANG’s abstractions are parameterized by domain. In the NEU design, this is an OCaml-style module functor:

```
module MakeCgen(Domain: ComponentDomain) { ... }
```

In the TypeScript implementation, this becomes a generic interface:

```
interface ComponentDomain<C, Constraints> {
  validate(component: C, constraints: Constraints): Result<C, string>;
  random(constraints: Constraints): C;
  mutate(component: C, constraints: Constraints): C;
  cross(a: C, b: C, constraints: Constraints): C;
  compare(a: C, b: C): number;
}
```

Swapping the domain from visual design to biomedicine requires implementing this interface for molecular structures—the evolutionary framework remains unchanged.

6.4 Provenance as Infrastructure

Every action in the CGEN-DLANG loop generates a provenance record: who contributed, what was created, what was selected, who voted, and how candidates were transformed. This is not logging—it is infrastructure. In the biomedical domain, provenance is the mechanism by which attribution and compensation are enforced.

7 Microfrontend Module System

The visual interface is implemented as a pipeline of composable microfrontend modules. Each module is a Web Component (native custom element) that encapsulates its own rendering, state, and typed ports.

7.1 Module Architecture

Every module declares:

- A **manifest** specifying its AIE engine type, language layer, and version
- Typed **input ports** and **output ports**
- A **visual interface** rendered in its Shadow DOM

Modules communicate through a **MessageBus**—a publish-subscribe system where messages carry data, timestamps, and provenance chains. When a module emits data on an output port, the pipeline routes it to the connected module’s input port, appending the source module to the provenance chain.

7.2 Pipeline Composition

The pipeline is wired declaratively:

```
pipeline.connect("component-library", "components",
                 "grammar-validator", "components");
pipeline.connect("grammar-validator", "valid-components",
                 "evolution-engine", "valid-components");
pipeline.connect("evolution-engine", "evolved",
                 "spectral-pipeline", "evolved");
pipeline.connect("spectral-pipeline", "results",
                 "cohort-analysis", "results");
```

This mirrors the AIE engine composition $f_1 \rightarrow \dots \rightarrow f_n$ and R’s pipe operator $|>$. The visual layout makes the data flow visible: users see data move from one module to the next.

7.3 Implemented Modules

Module	Engine Type	Language Layer	Function
Component Library	Synthesis	TypeScript	Generate molecular candidates
Grammar Validator	Transformation	OCaml	Validate biochemical constraints
Evolution Engine	Application	TypeScript	Community-guided evolutionary loop
Spectral Pipeline	Analysis	TypeScript	Disease spectroscopy classification
Cohort Analysis	Analysis	R	Statistical summaries and visualization
Provenance Tracker	Transformation	TypeScript	Attribution chain across pipeline

Table 4: Microfrontend modules in the pipeline, classified by AIE engine type and language layer.

Each module is self-contained and independently deployable. The Grammar Validator is tagged as the OCaml layer (ready to be replaced with Melange-compiled OCaml). The Cohort Analysis is tagged as the R layer (ready to be backed by WebR or Arrow). TypeScript orchestrates the connections.

7.4 Visual Design

Modules are visually distinguished by engine type (border color: purple for synthesis, blue for analysis, green for transformation) and language layer (badge color: orange for OCaml, TypeScript blue, R blue). Port indicators show input (cyan) and output (pink) connections. The pipeline layout is horizontal, with arrow connectors between modules, making the data flow architecture immediately legible.

8 Application: Equitable Biomedicine

The architecture is demonstrated in the domain of participatory personalized medicine and drug discovery. This domain demands exactly the properties the system provides: formal safety guarantees, community governance, data sovereignty, and traceable provenance.

8.1 The Detection Pipeline

The disease spectroscopy pipeline models biomarker detection for three disease categories:

Disease	Method	Key Biomarkers	AIE Type
Dementia	ATR-FTIR	A β 42 β -sheet (1630–1640 cm ⁻¹), phospho-tau, neurofilament light	Analysis
Diabetes	Mid-IR/NIR	Glucose (1035 cm ⁻¹), HbA1c, triglycerides, AGEs	Analysis
Respiratory	FTIR/SERS	Exhaled NO, breath acetone, IL-6 cytokine, surfactant lipids	Analysis

Table 5: Spectroscopic biomarker detection pipelines classified by AIE engine type.

Each pipeline follows: biological sample $\rightarrow$ spectroscopic measurement $\rightarrow$ preprocessing $\rightarrow$ ML classification $\rightarrow$ clinical output. In AIE terms: the measurement and classification stages are analysis

engines (reducing a complex biological sample to a risk score), and the preprocessing stage is a transformation (reformatting without information loss).

8.2 Extending cgen-dlang to Drug Discovery

The CGEN-DLANG framework maps directly from the design domain to the biomedical domain:

cgen-dlang Concept	Design Domain	Biomedical Domain
Component Library	Pre-defined visual assets	Molecular structures, biomarker signatures
Component Grammar	Rules for valid design operations	Lipinski’s Rule of Five, AlphaFold constraints
Voting Model	Community aesthetic preference	Community governance of research priorities
Evolutionary Operators	compare, mutate, cross	Iterative refinement of drug candidates
Fitness Function	Collective aesthetic judgment	Patient outcomes + efficacy + community values

Table 6: Mapping CGEN-DLANG abstractions from design to personalized medicine.

8.3 Governance Principles

The architecture enforces seven governance principles through its type system and module structure:

1. **Data stays with the person.** The **Federated** effect type ensures computation goes to the data, not the reverse. The **own** keyword tracks data ownership.
2. **Transparent provenance.** Every module emits provenance records on its output port. The Provenance Tracker module aggregates the full attribution chain.
3. **Collective governance.** The CGEN-DLANG voting model is the mechanism by which communities shape the evolutionary fitness function.
4. **Evolutionary co-design.** Drug candidates and treatment protocols are iteratively refined through community feedback loops, not top-down clinical trial structures.
5. **Attribution as infrastructure.** Provenance tracking is not an afterthought—it is a typed effect in the pipeline. A module cannot process data without emitting a provenance record.
6. **Compensation flows back.** When a model uses contributed data, the compensation effect fires. This is visible in the type signature and cannot be silently omitted.
7. **Right to withdraw.** Ownership semantics propagate revocation. If a participant withdraws consent, the type system propagates that constraint through the pipeline.

8.4 The Full Stack

Integrating all components, the architecture consists of six layers:

Layer 0: Biology

People, their biomarkers, their lived experience.

Layer 1: Detection

Spectroscopic measurement (ATR-FTIR, Mid-IR, NIR, SERS). Governed by data sovereignty frameworks.

Layer 2: Computation

Molecular dynamics, AlphaFold, HPC. Governed by federated learning and differential privacy.

Layer 3: Classification

ML pipeline: spectral preprocessing → feature extraction → classification. Governed by community-defined thresholds.

Layer 4: Governance

CGEN-DLANG extended: data cooperatives, collective voting, evolutionary optimization, provenance tracking.

Layer 5: Outcomes

Results returned to participants, revenue sharing, ongoing relationship, right to withdraw.

9 Related Work

9.1 Polyglot and Multi-Language Systems

The challenge of connecting languages with different type systems and runtime models is well-studied. Foreign function interfaces (FFIs) are the traditional approach, but they typically sacrifice type safety at the boundary. Melange [5] and js_of_ocaml [6] compile OCaml to JavaScript while preserving OCaml’s type guarantees. WebR [7] compiles R to WebAssembly, enabling R execution in browser and Node.js environments. Apache Arrow [8] provides a language-independent columnar memory format for zero-copy data exchange. Our architecture combines these bridges through a shared theoretical framework (AIE) that classifies computations regardless of implementation language.

9.2 Effect Systems and Typed Side Effects

Haskell’s monadic IO [9] demonstrated that side effects can be tracked in the type system. Algebraic effect systems [10] generalize this to user-defined effects. Koka [11] implements row-polymorphic effects. Our approach approximates effect tracking in TypeScript using branded types—a pragmatic choice that provides documentation and intent signaling without full compiler support.

9.3 Participatory Design in Computing

Participatory Design originated in Scandinavian workplace democracy movements [12]. Radical Participatory Design [2] extends this to challenge foundational hierarchies in research. The CGEN-DLANG framework [1] applies participatory and evolutionary principles to design collaboration. Our contribution extends CGEN-DLANG from visual design to biomedical applications, where the stakes of governance failures are measured in health outcomes rather than aesthetic preferences.

9.4 Evolutionary and Collective Intelligence

Interactive Evolutionary Algorithms (IEAs) combine computational generation with human evaluation [3]. Conversational Swarm Intelligence [4] enables networked groups to function as collective

decision-makers. The CGEN-DLANG framework’s voting model is an instance of this pattern, applied to the fitness function of an evolutionary algorithm.

9.5 Closest Language Designs

Several existing languages approximate aspects of the NEU design:

- **F#:** OCaml lineage with .NET ecosystem reach. Strong type system and data processing, but lacks R’s statistical primitives and Rust’s ownership model.
- **Scala:** Functional programming on the JVM with a powerful type system. Ecosystem is strong but complexity is high.
- **Julia:** Data science performance with multiple dispatch. Closest to NEU’s data-first philosophy, but lacks OCaml’s module system and Rust’s memory safety.
- **Gleam:** ML-family language targeting Erlang VM and JavaScript. Clean design but early ecosystem.

None of these combine all five pillars: expression orientation, data primitives, algebraic safety, pragmatic FP with effects, and polyglot reach.

9.6 AI-Assisted Compiler Construction and Self-Hosting

Recent work at Anthropic [16] demonstrated that a large language model (Claude) could write a self-hosting C compiler—a compiler that compiles its own source code—targeting three instruction set architectures (x86, ARM, RISC-V). The compiler implements lexing, parsing, type checking, optimization, register allocation, and code generation. The system achieves self-hosting on ARM and RISC-V entirely, and on x86 with a single delegation to GCC for 16-bit real-mode code generation where physical size constraints (32KB boot sector limit) make the AI-generated output (60KB+ due to opcode prefix overhead) infeasible.

This work validates two principles central to our architecture. First, **compilation target constraints are physical, not abstract:** instruction encoding overhead, binary size limits, and hardware fabric constraints are real boundaries that determine where delegation is necessary. Second, **self-hosting is a completeness test:** a system that can describe and produce itself demonstrates sufficient expressive power. For NEU, the analogous milestone is an AIE framework that can classify its own pipeline stages—which the microfrontend module system already partially achieves, as each module declares its own engine type.

9.7 OCaml for Hardware Description

Hardcaml [17] demonstrates that OCaml’s type system and module functors are effective for hardware description. Jane Street uses Hardcaml in production for FPGA-based trading systems, and the tool has been used to win cryptographic computation competitions (ZPrize). The Eclat project [18] extends this further, implementing a synchronous functional language that compiles to hardware and can host an OCaml virtual machine on FPGA. These projects demonstrate that OCaml’s reach extends from formal verification to physical hardware—a span that is uniquely relevant to a system like NEU that must bridge from spectroscopy hardware to governance UI.

9.8 Compiler-Driven UI Frameworks

Svelte [19] pioneered the compiler-driven approach to UI frameworks, shifting reactivity resolution from runtime (virtual DOM diffing) to compile time (direct DOM mutation). This philosophy—move work from runtime to build time—is shared with OCaml’s approach to type safety and with the broader NEU principle that formal properties should be verified before execution, not during it. The combination of Svelte (compile-time UI) with Melange (compile-time types) creates a frontend stack where both correctness and performance are addressed at build time.

10 Conclusion

We have presented NEU, a polyglot architecture that connects computational theory (AIE), language design (OCaml–TypeScript–R), and participatory infrastructure (CGEN-DLANG) into a coherent system for equitable biomedical computing.

The key contributions are:

1. **A formal connection between information-theoretic process classification and programming language type systems.** AIE’s engine taxonomy (synthesis, analysis, transformation) maps to traits and type constraints that can be checked at compile time. This is not merely a naming convention—it enables the compiler to verify that a pipeline stage classified as “analysis” actually reduces information content.
2. **A concrete polyglot architecture that leverages each language’s strengths.** OCaml provides the formal kernel (type safety, module functors, exhaustive validation). TypeScript provides the orchestration hub (async, UI, application logic). R provides the analytical layer (data frames, statistics, visualization). The architecture achieves approximately 80% of a purpose-built language’s guarantees using existing, mature tools.
3. **A microfrontend module system where visual interface components are typed computational engines.** Each module declares its AIE engine type, language layer, and typed ports. The pipeline composition is visible in the UI, making data flow and provenance legible to both developers and domain participants.

The application to equitable biomedicine demonstrates that these are not abstract concerns. When communities participate in drug discovery and disease spectroscopy, the formal properties—data sovereignty, provenance tracking, community governance, attribution, compensation—are the difference between participatory research and extractive data practices.

The architecture is implemented and running. The microfrontend pipeline compiles, serves, and demonstrates the full flow from candidate generation through grammar validation, evolutionary co-design, spectral analysis, cohort statistics, and provenance tracking. The OCaml and R layers are currently emulated in TypeScript with the correct type signatures and module boundaries, ready to be replaced with Melange-compiled OCaml and WebR-backed R as those bridges mature.

Future work includes: (1) implementing the OCaml kernel via Melange and measuring the type safety improvement at the boundary, (2) integrating WebR for real statistical computation in the Cohort Analysis module, (3) connecting the provenance system to blockchain-based attribution infrastructure, (4) field-testing the participatory voting model with a real community data cooperative, (5) replacing the Web Component rendering layer with Svelte to leverage compile-time UI optimization, and (6) prototyping Hardcaml-based FPGA acceleration for the spectral preprocessing pipeline, testing

whether OCaml’s module functors can parameterize hardware modules with the same domain interfaces used in the software pipeline.

The infrastructure should make participation the default, not the exception.

References

- [1] E.O. Oduniyi et al., “cgen.dlang: An evolutionary approach to design collaboration,” *AVI 2020: Data4Good Workshop*, 2020.
- [2] V. Udoewa, “An introduction to radical participatory design: decolonising participatory design processes,” *Design Science*, 2022. <https://doi.org/10.1017/dsj.2022.24>
- [3] J. He et al., “Collaborative intelligence in sequential experiments: A human-in-the-loop framework for drug discovery,” *Information Systems Research*, 2025. <https://doi.org/10.1287/isre.2024.1154>
- [4] L.B. Rosenberg, “Artificial swarm intelligence, a human-in-the-loop approach to A.I.,” *AAAI*, 2016/2023. <https://doi.org/10.1609/aaai.v30i1.9833>
- [5] Melange Contributors, “Melange: OCaml for JavaScript developers,” <https://melange.re>, 2023.
- [6] J. Vouillon and V. Balat, “From bytecode to JavaScript: the js_of_ocaml compiler,” *Software: Practice and Experience*, vol. 44, no. 8, pp. 951–972, 2014.
- [7] G. Sherlock, “WebR: R in the browser,” <https://docs.r-wasm.org/webr/latest/>, 2024.
- [8] Apache Arrow Contributors, “Apache Arrow: A cross-language development platform for in-memory analytics,” <https://arrow.apache.org>, 2022.
- [9] S. Peyton Jones, “Wearing the hair shirt: a retrospective on Haskell,” Invited talk, POPL 2003.
- [10] G. Plotkin and M. Pretnar, “Handlers of algebraic effects,” *ESOP 2009*, LNCS 5502, pp. 80–94, 2009.
- [11] D. Leijen, “Koka: Programming with row polymorphic effect types,” *MSFP 2014*, EPTCS 153, pp. 100–126, 2014.
- [12] D. Schuler and A. Namioka, *Participatory Design: Principles and Practices*, Lawrence Erlbaum Associates, 1993.
- [13] M. Swan, “Health 2050: The realization of personalized medicine through crowdsourcing, the quantified self, and the participatory biocitizen,” *Journal of Personalized Medicine*, vol. 2, no. 3, pp. 93–118, 2012.
- [14] A.M. McCartney et al., “Benefit-sharing by design: A call to action for human genomics research,” *Annual Review of Genomics and Human Genetics*, 2024. <https://doi.org/10.1146/annurev-genom-021623-104241>
- [15] S.R. Carroll et al., “The CARE Principles for Indigenous Data Governance,” *Data Science Journal*, 2020. <https://doi.org/10.5334/dsj-2020-043>
- [16] Anthropic Engineering, “Building a C compiler in Claude’s extended thinking,” Anthropic Blog, 2025. <https://www.anthropic.com/engineering/building-c-compiler>

- [17] Jane Street, “Hardcaml: An OCaml library for designing hardware,” <https://github.com/janestreet/hardcaml>, 2023.
- [18] L. Music and L. Music, “Eclat: A synchronous functional language for programming FPGAs,” *ACM SIGPLAN Workshop on Partial Evaluation and Program Manipulation*, 2023.
- [19] R. Harris, “Svelte 3: Rethinking reactivity,” <https://svelte.dev/blog/svelte-3-rethinking-reactivity>, 2019.
- [20] T.M. Cover and J.A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley-Interscience, 2006.
- [21] N. Tishby, F.C. Pereira, and W. Bialek, “The information bottleneck method,” *Proc. 37th Annual Allerton Conference on Communication, Control, and Computing*, pp. 368–377, 1999.
- [22] R. Shwartz-Ziv and N. Tishby, “Opening the black box of deep neural networks via information,” *arXiv:1703.00810*, 2017.
- [23] F. Bonchi, P. Sobociński, and F. Zanasi, “A categorical semantics of signal flow graphs,” *CONCUR 2014*, LNCS 8704, pp. 435–450, 2014. https://doi.org/10.1007/978-3-662-44584-6_30
- [24] B. Fong and D.I. Spivak, *An Invitation to Applied Category Theory: Seven Sketches in Compositionality*, Cambridge University Press, 2019. <https://doi.org/10.1017/9781108668804>
- [25] J. Foley, S. Breiner, E. Subrahmanian, and J. Dusel, “Operads for complex system design specification, analysis and synthesis,” *Proc. Royal Society A*, vol. 477, no. 2252, 2021. <https://doi.org/10.1098/rspa.2021.0099>
- [26] W. Yang et al., “A mathematical framework of semantic communication based on category theory,” *arXiv:2504.11334*, 2025.